Q. explain abstraction, encapsulation, polymorphism, inheritance in short with short programs in java.

Ans:

1. Abstraction

Definition: Hiding internal details and showing only essential features.

✅ Achieved using **abstract classes** or **interfaces**.

java
CopyEdit
```
abstract class Animal {
  abstract void sound(); // abstract method
}

class Dog extends Animal {
  void sound() {
    System.out.println("Barks");
  }
}

public class Main {
  public static void main(String[] args) {
    Animal a = new Dog();
    a.sound(); // Output: Barks
  }
}
```

---

2. Encapsulation

Definition: Wrapping data (variables) and methods into a single unit (class), and restricting access via access modifiers.

✅ Achieved using **private fields + public getters/setters**.

java
CopyEdit
```
class Person {
  private String name;

  public void setName(String n) {
    name = n;
  }

  public String getName() {
    return name;
  }
}

public class Main {
  public static void main(String[] args) {
    Person p = new Person();
    p.setName("Alice");
    System.out.println(p.getName()); // Output: Alice
  }
}
```

---

3. Polymorphism

Definition: One function behaves differently for different objects.

✅ Achieved by **method overriding** (runtime) or **method overloading** (compile time).

Method Overriding:

java
CopyEdit
```
class Animal {
  void sound() {
    System.out.println("Some sound");
  }
}

class Cat extends Animal {
  void sound() {
    System.out.println("Meows");
  }
}

public class Main {
  public static void main(String[] args) {
    Animal a = new Cat();
    a.sound(); // Output: Meows
  }
}
```

---

4. Inheritance

Definition: One class (child) inherits fields and methods from another (parent).

✅ Achieved using the **extends** keyword.

java
CopyEdit
```
class Vehicle {
  void run() {
    System.out.println("Vehicle is running");
  }
}

class Bike extends Vehicle {
  // inherits run()
}

public class Main {
```

```
    public static void main(String[] args) {
        Bike b = new Bike();
        b.run(); // Output: Vehicle is running
    }
}
```

Q. explain abstract class and interfaces
Ans:
Abstract Class

- A class that **cannot be instantiated**.
- Can have **abstract methods** (no body) and **concrete methods** (with body).
- Use when classes **share common behavior**.

```
java
CopyEdit
abstract class Animal {
    abstract void sound(); // abstract method

    void eat() {          // concrete method
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Barks");
    }
}
```

Key Points:

- Use abstract keyword.
- Can have fields and constructors.
- Supports partial implementation.

✅ **Interface**

- A **contract** that a class agrees to follow.
- Contains **only abstract methods** (Java 7), or abstract + default/static methods (Java 8+).
- A class **implements** an interface.

```
java
CopyEdit
interface Animal {
    void sound(); // implicitly public and
abstract
}

class Cat implements Animal {
    public void sound() {
        System.out.println("Meows");
```

Q. explain garbage collector in java in short with code.
Ans.
The **Garbage Collector (GC)** in Java automatically **reclaims memory** used by objects that are **no longer reachable**—so you don't have to manually delete objects like in C/C++.

```
public class GarbageDemo {
    public void finalize() {
        System.out.println("Garbage collected");
    }

    public static void main(String[] args) {
        GarbageDemo obj1 = new
GarbageDemo();
        GarbageDemo obj2 = new
GarbageDemo();

        obj1 = null; // Now eligible for GC
        obj2 = null;

        System.gc(); // Suggests GC to JVM
    }
}
```

 When System.gc() is called, the JVM may run the garbage collector.
 The finalize() method (if overridden) is called **before** an object is garbage collected (though it's deprecated in modern Java).

Q. explain lambda expression in java
Ans.

```
@FunctionalInterface
interface Greet {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {
        Greet g = () -> System.out.println("Hello
from Lambda!");
        g.sayHello(); // Output: Hello from
Lambda!
    }
}
```

 Less boilerplate code
 Works well with collections & streams
 Improves readability and conciseness

Q. explain multithreading in java in short
Ans.
Multithreading allows **multiple threads** to run **concurrently** in a Java program, improving performance especially for tasks like I/O, games, or UI.
Java uses the Thread class or implements the Runnable interface to create threads.
Extending Thread class

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running");
   }

   public static void main(String[] args) {
      MyThread t = new MyThread();
      t.start();  // starts the thread
   }
}
```

Implementing Runnable interface

```
class MyRunnable implements Runnable {
   public void run() {
      System.out.println("Runnable thread
running");
   }

   public static void main(String[] args) {
      Thread t = new Thread(new
MyRunnable());
      t.start();
   }
}
```

Q. explain thread life cycle in java
Ans.
A thread in Java goes through several states from creation to termination.

1. New

- **Thread is created using new Thread(), but not yet started.**

Thread t = new Thread();

🔁 2. Runnable

- **Thread is ready to run, waiting for CPU.**
- **Entered using start() method.**

t.start();

🔁 3. Running

- **Thread is executing its run() method.**
- **JVM scheduler picks it from runnable threads.**

🔁 4. Blocked / Waiting / Timed Waiting

- **Blocked: Waiting to access a locked resource.**
- **Waiting: Waiting indefinitely (e.g., join()).**
- **Timed Waiting: Waiting for a specified time (e.g., sleep(1000)).**

🔁 5. Terminated (Dead)

- **Thread has finished execution or exited due to an error.**

```
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running");
   }

   public static void main(String[] args) {
      MyThread t = new MyThread();
      t.start();
   }
}
```

Q. explain exception handling in java in short and types of exception
Ans.
Exception Handling in Java is used to handle **errors** during program execution **without crashing** the program.

Basic Syntax (Try-Catch-Finally)

```
try {
   // Code that may throw an exception
   int result = 10 / 0;
} catch (ArithmeticException e) {
   System.out.println("Cannot divide by zero");
} finally {
   System.out.println("This always runs");
}
```

Types of Exceptions in Java
1. Checked Exceptions

- **Checked at compile-time**
- Must be handled with try-catch or throws

```
import java.io.*;

try {
```

```
    FileReader fr = new FileReader("file.txt");
} catch (FileNotFoundException e) {
    System.out.println("File not found");
}
```

2. Unchecked Exceptions

- **Happen at runtime**
- **Not** checked by compiler

```
int[] arr = {1, 2};
System.out.println(arr[5]); // throws ArrayIndexOutOfBoundsException
```

Q. explain keywords in java in short
Ans.
Java keywords are **reserved words** used by the Java language that have **predefined meaning**. You **cannot use them as variable or method names**.

- abstract
- assert
- boolean
- break
- byte
- case
- catch
- char
- class
- const

```
public class Car {
    private String brand;
    static int count = 0;

    public Car(String brand) {
        this.brand = brand;
        count++;
    }

    public void show() {
        System.out.println("Brand: " + brand);
    }

    public static void main(String[] args) {
        Car c1 = new Car("Toyota");
        c1.show();
    }
}
```

Q. explain fundamentals of java servlet programming
Ans.
A **Java Servlet** is a **server-side program** that runs on a **Java-enabled web server** (like Apache Tomcat). It is used to **handle requests** and **generate dynamic web content** (like HTML, JSON, etc.).

- **Servlet Container (Web Server)**
  - The server (e.g., Tomcat) that runs and manages servlets.
  - It manages lifecycle, request handling, and response generation.
- **Servlet Lifecycle**
  - **Initialization (init())**: Called once when the servlet is created.
  - **Request Handling (service())**: Handles each request sent to the servlet.
  - **Destruction (destroy())**: Called once when the servlet is removed from service.
- **Servlet API**
  - **HttpServlet**: A common base class to handle HTTP requests (GET, POST, etc.).
  - **ServletRequest** and **ServletResponse**: Used to handle client request and send response.

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<h1>Hello, World!</h1>");
    }
}
```

Q. explain servlet life cycle in java
Ans.
The **Servlet Life Cycle** defines the steps a servlet goes through from creation to destruction. These steps are managed by the **Servlet Container** (e.g., Tomcat).

Key Methods in Servlet Life Cycle:

- **init()**: Called once when the servlet is created, for initialization.
- **service()**: Called for each request, handles the logic.
- **destroy()**: Called when the servlet is removed from service.

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class HelloServlet extends HttpServlet {
   // Initialization
   public void init() {
      System.out.println("Servlet Initialized");
   }

   // Request Handling
   protected void doGet(HttpServletRequest
request, HttpServletResponse response)
throws ServletException, IOException {
      PrintWriter out = response.getWriter();
      out.println("Hello, Servlet!");
   }

   // Destruction
   public void destroy() {
      System.out.println("Servlet Destroyed");
   }
}
```

Q. explain working with cookies
Ans.
A **cookie** is a small piece of data stored on the client-side (browser) and sent with every request to the server. Cookies are used to **store user preferences**, **login sessions**, etc.

Key Operations:

1. **Creating a Cookie**
2. **Sending Cookies to Client**
3. **Reading Cookies from Client**

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class CookieExample extends
HttpServlet {
   protected void doGet(HttpServletRequest
request, HttpServletResponse response)
throws ServletException, IOException {
      // Create a cookie
      Cookie cookie = new Cookie("username",
"john_doe");
      cookie.setMaxAge(60 * 60); // 1 hour
      response.addCookie(cookie); // Send
cookie to client

      // Read the cookie
      Cookie[] cookies = request.getCookies();
      if (cookies != null) {
         for (Cookie c : cookies) {
            if (c.getName().equals("username")) {
               String username = c.getValue();

response.getWriter().println("Welcome back,
" + username);
            }
         }
      }
   }
}
```

- Cookies are **sent with every request** to the same server until they expire.
- **Secure** cookies should be used for sensitive data.
- **Cookies have a size limit** (around 4 KB).

Q. explain JSP tags and components of JSP
Ans.
JSP Tags:

1. **<%@ page %>**
2. **<%@ include %>**
3. **<%@ taglib %>**
4. **<%! %>**
5. **<% %>**

JSP Components:

1. **JSP Page (View)**
2. **JSP Engine (Container)**
3. **JavaBeans**
4. **Custom Tags**
5. **Tag Libraries**

```
<%@ page contentType="text/html;
charset=UTF-8" language="java" %>
<html>
<body>
  <h2>Welcome to JSP</h2>
  <form action="greet.jsp" method="post">
    Enter your name: <input type="text"
name="username" />
    <input type="submit" value="Submit" />
  </form>
  <%
    String name =
request.getParameter("username");
    if (name != null) {
      out.println("<p>Hello, " + name +
"!</p>");
    }
  %>
</body>
</html>
```

JavaServer Pages (JSP) is a technology that allows you to embed Java code directly into HTML pages. It's a part of Java EE (Enterprise Edition) and used to create dynamic web pages.

JSP Components: Used to represent Java objects that can be used in JSP pages to manage data. They are set and retrieved using JSP tags like <jsp:setProperty /> and <jsp:getProperty />.

Q. explain expressions scriptlets directives and JSP object
Ans.

JSP Expressions

- **Purpose**: Used to output the result of a **Java expression** directly to the client (HTML).
- **Syntax**: <%= expression %>
- **Behavior**: The expression inside <%= %> is evaluated, and the result is inserted into the HTML content.

JSP Scriptlets

- **Purpose**: Allows embedding **Java code** into the JSP page.
- **Syntax**: <% java code %>
- **Behavior**: The Java code inside the scriptlet is executed when the page is requested.

JSP Directives

- **Purpose**: Provides **page-level settings** to the JSP container (server).
- **Syntax**: <%@ directive %>
- **Common Types**:
  - <%@ page %>: Defines page-level attributes like content type, language, etc.
  - <%@ include %>: Includes another file (static content).
  - <%@ taglib %>: Declares a tag library for custom tags.

JSP Objects

JSP provides several **implicit objects** that are automatically available within the JSP page. They are used to interact with the client's request, session, and other aspects.

Common JSP Objects:

1. **request**: Represents the HTTP request.
2. **response**: Represents the HTTP response.
3. **out**: Used to send output to the client.
4. **session**: Represents the HTTP session.
5. **application**: Represents the ServletContext (global application data)

Summary:

- **Expressions**: Output the result of Java code.
- **Scriptlets**: Embed Java code in the JSP.

- **Directives**: Provide page configuration.
- **JSP Objects**: Implicit objects like request, response, session, etc.

Q. explain JDBC connectivity in java
Ans.
JDBC (Java Database Connectivity) is a standard API for connecting Java applications to databases, allowing them to execute SQL queries and retrieve results.

Steps for JDBC Connectivity:

1. Import JDBC packages.
2. Load the database driver.
3. Establish a connection to the database.
4. Create a statement object.
5. Execute SQL queries (e.g., executeQuery(), executeUpdate()).
6. Process the results (using ResultSet).
7. Close the resources (ResultSet, Statement, Connection).

Q. explain spring MVC architecture in java
Ans.
Spring MVC (Model-View-Controller) is a design pattern used in Spring Framework for developing web applications. It separates the application into three main components: **Model**, **View**, and **Controller**.

Components of Spring MVC:

1. **Model**
    - Represents the **data** of the application.
    - It can be any object (like JavaBeans, POJOs) that holds the application's state or business logic.
2. **View**
    - Represents the **UI** or the **output** that the user sees (e.g., JSP, Thymeleaf).
    - It is responsible for rendering the model data and presenting it to the user.
3. **Controller**
    - Acts as an **intermediary** between the Model and the View.
    - It processes incoming requests, manipulates the model, and returns the appropriate view.

How Spring MVC Works (Flow):

1. **Client Request**
    - The user sends an HTTP request to the server.
2. **DispatcherServlet**
    - The **DispatcherServlet** is the front controller that intercepts all requests.
    - It delegates the request to the appropriate **Controller**.
3. **Controller**
    - The **Controller** processes the user request and returns a **ModelAndView** object.
    - It contains the data (model) and the name of the view to be rendered.
4. **View Resolver**
    - The **View Resolver** resolves the view name provided by the controller to a specific view (like a JSP or Thymeleaf template).
5. **View Rendering**
    - The view renders the data (model) to the client.

Q. explain spring MVC annotation in java
Ans.
Spring MVC uses annotations to simplify the configuration and wiring of controllers, request mappings, and other components.

 @Controller: Marks a class as a controller.
 @RequestMapping: Maps URLs to methods.
 @GetMapping, @PostMapping, etc.: Shorthand for specific HTTP methods.
 @PathVariable: Binds path variables from the URL.
 @RequestParam: Binds query parameters.
 @ModelAttribute: Binds form data to an object.
 @ResponseBody: Returns data directly in the response body.
 @RequestBody: Maps request body to a Java object.

```
// 1. Controller class
@Controller
public class UserController {

   // 2. GET request - show user page
   @GetMapping("/user")
   public String showUserPage() {
      return "user";  // JSP/HTML page name
   }

   // 3. GET request with path variable
   @GetMapping("/user/{id}")
   @ResponseBody
   public String getUser(@PathVariable int id) {
      return "User ID: " + id;
   }

   // 4. POST request with form data
   @PostMapping("/saveUser")
   public String saveUser(@ModelAttribute
User user) {
      // save user logic here
      return "success";
   }

   // 5. POST with JSON body (REST API)
   @PostMapping("/api/user")
   @ResponseBody
   public String createUser(@RequestBody
User user) {
      return "Created user: " + user.getName();
   }
}
```

Q. explain MVC flow in java
Ans.
MVC (Model-View-Controller) is a design pattern that separates an application into 3 layers to organize code better.

1. Client (Browser) Sends Request

- User clicks a link or submits a form.

🔷 2. Controller (Handles Request)

- The request is sent to the **Controller** (Java class with @Controller).
- It processes the request, prepares data (Model), and selects a View.

🔷 3. Model (Data/Business Logic)

- **Model** contains the data or logic (like JavaBeans or DB results).
- The Controller adds data to the Model.

🔷 4. View (Response Page)

- The **View** (like JSP/HTML) takes the data from the Model and displays it to the user.

🔁 Flow Diagram (Simple)

User → Controller → Model → View → User

Example Flow:

1. User opens http://site.com/user/1
2. Controller handles /user/{id}
3. Controller fetches user data (Model)
4. Passes data to JSP page (View)

Q. explain spring form handling in java

Ans.

Spring MVC simplifies **form handling** using @ModelAttribute, form tags, and binding objects.

Basic Flow:

1. **Form Page (HTML/JSP)**
    - Displays a form to the user.
    - Uses Spring form tags like <form:form> and <form:input>.
2. **Model Object (JavaBean)**
    - A simple Java class to hold form data (e.g., User).
3. **Controller**
    - Shows the form using @GetMapping
    - Handles form submission using @PostMapping and @ModelAttribute
4. **Result Page**
    - Displays success message or result.

```
@Controller
public class UserController {

   @GetMapping("/form")
   public String showForm(Model model) {
      model.addAttribute("user", new User());
      return "userForm";
   }

   @PostMapping("/submitForm")
   public String
submitForm(@ModelAttribute("user") User
user) {
      // process user data
      return "formSuccess";
   }
}
```

Summary:

- @ModelAttribute binds form data to a model object.
- <form:form> and <form:input> help create dynamic, data-bound forms.
- Controller handles both form display and submission.

Q. explain spring core, springboot dependency injection, and Inversion of control(IOC)

Ans.

Spring Core is the **foundation** of the Spring Framework. It provides the basic features for building Java applications, like managing objects and their dependencies.

Key Concepts:

1. **Dependency Injection (DI)**
    - Spring creates and manages objects for you.
    - It "injects" dependencies instead of you creating them manually using new.
2. **Inversion of Control (IoC)**
    - You give control to the Spring container to manage objects (beans).
    - You configure what you want, and Spring does the wiring.
3. **Beans**
    - Objects managed by Spring container.
    - Defined in XML or with annotations like @Component.
4. **ApplicationContext**
    - A container that holds and manages beans.
    - Loads configuration and provides access to beans.

Summary:

- **Spring Core** handles object creation and wiring (DI, IoC).
- **Beans** are Java objects managed by Spring.
- **ApplicationContext** is the container that manages them.

Simple Spring Core Example (Annotation-based)

java
CopyEdit

```
// 1. A Bean Class
import
org.springframework.stereotype.Component;

@Component
public class HelloService {
   public void sayHello() {
      System.out.println("Hello from Spring
Core!");
   }
}
```

Main Class:

```
import
org.springframework.context.ApplicationCont
ext;
import
org.springframework.context.annotation.Ann
otationConfigApplicationContext;

public class MainApp {
   public static void main(String[] args) {
      ApplicationContext context = new
AnnotationConfigApplicationContext(AppConf
ig.class);
      HelloService hello =
context.getBean(HelloService.class);
      hello.sayHello();
   }
}
```

Configuration class

```
// 2. Configuration Class
import
org.springframework.context.annotation.Com
ponentScan;
import
org.springframework.context.annotation.Confi
guration;

@Configuration
@ComponentScan(basePackages =
"com.example")  // change to your package
name
public class AppConfig {
```